VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Pruthviraj Yadav
(1WA24CS224)

in partial fulfillment for the award of the degree of
BACHELOROFE NGINEERING
in
COMPUTER SCIENCEAND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
BullTempleRoad,Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Pruthviraj Yadav **(1WA24CS224)**, who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026
. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. SeemaPatil
AssistantProfessor
Department of CSE
BMSCE,Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

CODE:

```
#include <stdio.h>

int main() {
   int n, i, choice;
   intat[10],bt[10],ct[10],tat[10], wt[10];

   printf("Enternumberofprocesses: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++) {
      printf("EnterATandBTfor P%d: ", i+1);
      scanf("%d%d",&at[i],&bt[i]);
   }

   printf("\nSelect Type:\n");
   printf("1.Non-Preemptive\n");
   printf("2. Preemptive\n");
   printf("Enter choice: ");
   scanf("%d", &choice);

   if(choice == 1) {
      // FCFS Scheduling
      ct[0] = at[0] + bt[0];

      for(i = 1; i < n; i++) {
         if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
         else
            ct[i] = ct[i-1] + bt[i];
      }

      for(i = 0; i < n; i++) {
         tat[i] = ct[i] - at[i];
         wt[i] = tat[i] - bt[i];
      }

      printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
      for(i = 0; i < n; i++) {
         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
         i+1,at[i],bt[i],ct[i],tat[i], wt[i]);
      }
```

```
    }
    else {
        printf("FCFS does not support Preemptive scheduling\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 2
Enter AT and BT for P1: 0 5
Enter AT and BT for P2: 0 4

Select Type:
1. Non-Preemptive
2. Preemptive
Enter choice: 1

P       AT      BT      CT      TAT     WT
P1      0       5       5       5       0
P2      0       4       9       9       5

Process returned 0 (0x0)   execution time : 28.946 s
Press any key to continue.
```

b) SJF (Preemptive and Non-Preemptive)

CODE:

```
#include <stdio.h>
#include <limits.h>

int main() {
   int n,i,time=0,done=0,choice;
   int at[10],bt[10],rt[10],ct[10],wt[10],tat[10];

   printf("Enter number of processes: ");
   scanf("%d",&n);

   for(i=0;i<n;i++){
      printf("AT and BT of P%d: ",i+1);
      scanf("%d%d",&at[i],&bt[i]);
      rt[i]=bt[i];
   }

   printf("\nSelect Type:\n");
   printf("1. Non-Preemptive\n");
   printf("2. Preemptive\n");
   printf("Enter choice: ");
   scanf("%d", &choice);

   if(choice==1){ // Non-preemptive
      int  vis[10]={0};
      while(done<n){
         int min=INT_MAX,p=-1;
         for(i=0;i<n;i++)
            if(at[i]<=time && !vis[i] && bt[i]<min){
               min=bt[i]; p=i;
            }
         if(p==-1){
         time++;
         continue;
         }

         time+=bt[p];
         ct[p]=time;
         vis[p]=1;
         done++;
      }
   }
   else{ // Preemptive (SRTF)
      while(done<n){
         int min=INT_MAX,p=-1;
         for(i=0;i<n;i++)
```

```
            if(at[i]<=time && rt[i]>0 && rt[i]<min){
               min=rt[i]; p=i;
            }
         if(p==-1){
         time++;
         continue;
         }

         rt[p]--;
         time++;

         if(rt[p]==0){
            ct[p]=time;
            done++;
         }
      }
   }

   float  tat_total=0;
   float wt_total=0;

   printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
   for(i=0;i<n;i++){
      tat[i]=ct[i]-at[i];
      tat_total+=tat[i];
      wt[i]=tat[i]-bt[i];
      wt_total+=wt[i];
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
   }
   printf("Average  tat:  %f",(tat_total)/n);
   printf("Average  wt:  %f",(wt_total)/n);
   return 0;
}
```

OUTPUT:

```
Enter number of processes: 2
AT and BT of P1: 0 5
AT and BT of P2: 0 4

Select Type:
1. Non-Preemptive
2. Preemptive
Enter choice: 1

P       AT      BT      CT      TAT     WT
P1      0       5       9       9       4
P2      0       4       4       4       0

Process returned 0 (0x0)   execution time : 12.355 s
Press any key to continue.
```

```
Enter number of processes: 2
AT and BT of P1: 0 5
AT and BT of P2: 0 4

Select Type:
1. Non-Preemptive
2. Preemptive
Enter choice: 2

P       AT      BT      CT      TAT     WT
P1      0       5       9       9       4
P2      0       4       4       4       0

Process returned 0 (0x0)   execution time : 9.342 s
Press any key to continue.
```

c) Priority (Preemptive and Non-Preemptive)

```
#include<stdio.h>
#include<stdlib.h>

intmain(){
   intn,i,choice,count=0,time=0;
   printf("Enter no. of processes: \n");
   scanf("%d",&n);

   intat[n],bt[n],pr[n],ct[n],tat[n],wt[n],rt[n];
   for(i=0;i<n;i++){
      printf("Enter AT,BT and Priority for P%d: ",i+1);
      scanf("%d %d %d",&at[i],&bt[i],&pr[i]);
      rt[i]=bt[i];
   }
   printf("Select Type:\n");
   printf("1. Non preemptive\n");
   printf("2. Preemptive\n");
   printf("Enter choice: ");
   scanf("%d",&choice);

   if(choice==1){
      intcompleted[n];
      for(i=0;i<n;i++)
         completed[i]=0;
```

```
    while(count<n){
        intidx=-1;
        inthighest=9999;

        for(i=0;i<n;i++){
            if(at[i]<=time && completed[i]==0){
                if(pr[i]<highest){
                    highest=pr[i];
                    idx=i;
                }
            }
        }

        if(idx!=-1){
            time+=bt[idx];

            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];

            completed[idx]=1;
            count++;
        }
        elsetime++;
    }
}

elseif(choice==2){
    while(count<n){
        intidx=-1;
        inthighest=9999;

        for(i=0;i<n;i++){
            if(at[i]<=time && rt[i]>0){
                if(pr[i]<highest){
                    highest=pr[i];
                    idx=i;
                }
            }
        }

        if(idx!=-1){
            rt[idx]--;
            time++;

            if(rt[idx]==0){
                ct[idx]=time;
                tat[idx]=ct[idx]-at[idx];
```

```
                wt[idx]=tat[idx]-bt[idx];
                count++;
            }
        }
        elsetime++;
    }
}

elseprintf("Enter from the choices given!\n");

printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\n");

floattatsum=0,wtsum=0;

for(i=0;i<n;i++){
tatsum+=tat[i];
wtsum+=wt[i];
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
}

printf("\nAvg TAT: %f\nAvg WT: %f\n",tatsum/n,wtsum/n);

return 0;
}
```

OUTPUT:

```
Enter no. of processes:
4
Enter AT,BT and Priority for P1: 0 4 5
Enter AT,BT and Priority for P2: 3 2 6
Enter AT,BT and Priority for P3: 5 5 1
Enter AT,BT and Priority for P4: 2 3 3
Select Type:
1. Non preemptive
2. Preemptive
Enter choice: 1

P       AT      BT      PR      CT      TAT     WT
P1      0       4       5       4       4       0
P2      3       2       6       14      11      9
P3      5       5       1       12      7       2
P4      2       3       3       7       5       2

Avg TAT: 6.750000
Avg WT: 3.250000

Process returned 0 (0x0)   execution time : 28.057 s
Press any key to continue.
```

```
Enter no. of processes:
4
Enter AT,BT and Priority for P1: 0 4 5
Enter AT,BT and Priority for P2: 3 2 6
Enter AT,BT and Priority for P3: 5 5 1
Enter AT,BT and Priority for P4: 2 3 3
Select Type:
1. Non preemptive
2. Preemptive
Enter choice: 2

P       AT      BT      PR      CT      TAT     WT
P1      0       4       5       12      12      8
P2      3       2       6       14      11      9
P3      5       5       1       10      5       0
P4      2       3       3       5       3       0

Avg TAT: 7.750000
Avg WT: 4.250000

Process returned 0 (0x0)   execution time : 26.453 s
Press any key to continue.
```

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

```
#include <stdio.h>
#define MAX 100

int main() {
   int n, q_count;
   int at[MAX], bt[MAX], rt[MAX];
   int ct[MAX], tat[MAX], wt[MAX];
   int queue[MAX], visited[MAX];
   int front, rear, time, completed;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter Arrival Time and Burst Time:\n");
   for (int i = 0; i < n; i++) {
      printf("P%d AT BT: ", i + 1);
      scanf("%d %d", &at[i], &bt[i]);
   }
   printf("\nEnter number of quantum values: ");
   scanf("%d", &q_count);
   int quantum[q_count];
   printf("Enter quantum values:\n");
   for (int i = 0; i < q_count; i++) {
   scanf("%d", &quantum[i]);
   }
   for (int q = 0; q < q_count; q++) {
      for (int i = 0; i < n; i++) {
         rt[i] = bt[i];
         visited[i] = 0;
      }
      front = rear = 0;
      time = 0;
      completed = 0;
      queue[rear++] = 0;
      visited[0] = 1;
      while (completed < n) {
         if (front == rear)
            break;
         int i = queue[front++];
         if (rt[i] > quantum[q]) {
            time += quantum[q];
            rt[i] -= quantum[q];
         }
         else {
            time += rt[i];
            rt[i] = 0;
            ct[i] = time;
            completed++;
```

```
            }
            for (int j = 0; j < n; j++) {
                if(!visited[j]&&at[j]<= time) {
                    queue[rear++] = j;
                    visited[j] = 1;
                }
            }
            if (rt[i] > 0)
                queue[rear++] = i;
            if (front == rear) {
                for(intj=0;j<n;j++){
                    if (rt[j] > 0) {
                        queue[rear++] = j;
                        visited[j] = 1;
                        time = at[j];
                        break;
                    }
                }
            }
        }
        floattotal_tat=0,total_wt=0;
        printf("\nQuantum=%d\n",quantum[q]);
        printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
        for (int i = 0; i < n; i++) {
            tat[i] = ct[i] - at[i];
            wt[i] = tat[i] - bt[i];
            total_tat += tat[i];
            total_wt += wt[i];
            printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        }
        printf("AverageTAT=%.2f\n",  total_tat  /  n);
        printf("AverageWT=%.2f\n", total_wt / n);
    }
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT BT: 1 8
P2 AT BT: 4 7
P3 AT BT: 2 3

Enter number of quantum values: 3
Enter quantum values:
2
5
3

Quantum = 2
PID     AT      BT      CT      TAT     WT
P1      1       8       15      14      6
P2      4       7       18      14      7
P3      2       3       9       7       4
Average TAT = 11.67
Average WT  = 5.67

Quantum = 5
PID     AT      BT      CT      TAT     WT
P1      1       8       16      15      7
P2      4       7       18      14      7
P3      2       3       13      11      8
Average TAT = 13.33
Average WT  = 7.33

Quantum = 3
PID     AT      BT      CT      TAT     WT
P1      1       8       14      13      5
P2      4       7       18      14      7
P3      2       3       6       4       1
Average TAT = 10.33
Average WT  = 4.33

Process returned 0 (0x0)   execution time : 39.092 s
Press any key to continue.
```

2.Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

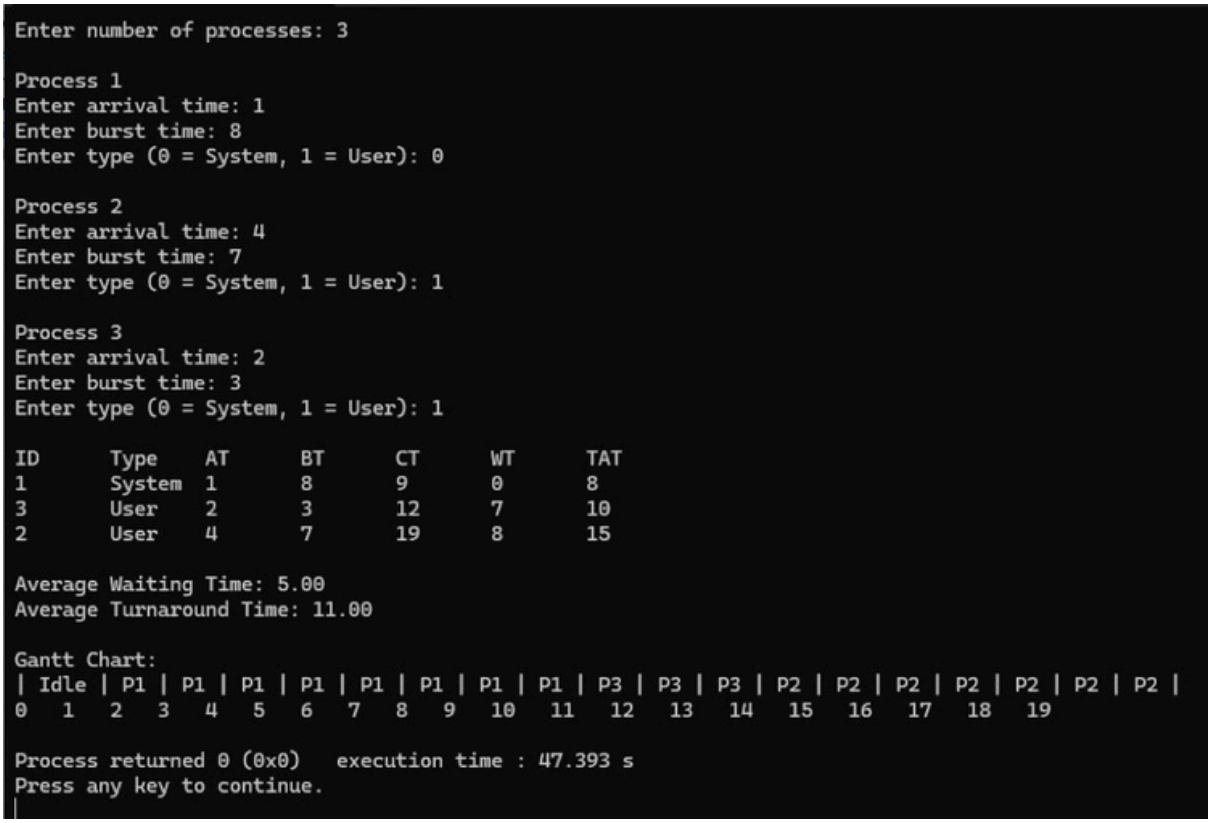
```
#include <stdio.h>

int main() {
   int n;
   printf("Enternumberofprocesses:");
   scanf("%d", &n);

   intpid[n],at[n],bt[n],rt[n],ct[n],wt[n], tat[n], type[n];

   for (int i = 0; i < n; i++) {
      printf("\nProcess %d\n", i+1);
      pid[i] = i+1;

      printf("Enter arrival time: ");
      scanf("%d", &at[i]);

      printf("Enter burst time: ");
      scanf("%d", &bt[i]);

      printf("Entertype(0=System,1= User): ");
      scanf("%d", &type[i]);

      rt[i] = bt[i];
   }
   for(inti=0;i<n-1;i++){
      for(intj=0;j<n-i-1;j++){
         if (at[j] > at[j + 1]) {
            int temp;
```

```
            temp = at[j];
            at[j] = at[j + 1];
            at[j + 1] = temp;
            temp = bt[j];
            bt[j] = bt[j + 1];
            bt[j + 1] = temp;
            temp = rt[j];
            rt[j] = rt[j + 1];
            rt[j + 1] = temp;
            temp = pid[j];
            pid[j] = pid[j + 1];
            pid[j + 1] = temp;
            temp = type[j];
            type[j] = type[j + 1];
            type[j + 1] = temp;
        }
    }
}
intsystemQ[100],userQ[100];
intsf=0,sr=0,uf=0,ur=0;
intcurrent_time=0,completed = 0, i = 0;
int current = -1;
int gantt[1000], gindex = 0;
while (completed < n) {
    while(i<n&&at[i]<=current_time) {
        if (type[i] == 0)
            systemQ[sr++] = i;
        else
            userQ[ur++] = i;
        i++;
    }
    if (current != -1) {
        if(type[current]==1&& sf < sr) {
            userQ[ur++]=current;
            current = -1;
        }
    }
    if (current == -1) {
        if (sf < sr)
            current=systemQ[sf++];
```

```
        else if (uf < ur)
            current=userQ[uf++];
        else {
            gantt[gindex++] = -1;
            current_time++;
            continue;
        }
    }
    gantt[gindex++]=pid[current];
    rt[current]--;
    current_time++;
    if(rt[current]==0){
        ct[current]=current_time;
        completed++;
        current = -1;
    }
}
floattotal_wt=0,total_tat = 0;
for(inti=0;i<n;i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    total_wt += wt[i];
    total_tat += tat[i];
}
printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
for(inti=0;i<n;i++) {
    if (type[i] == 0)
        printf("%d\tSystem\t", pid[i]);
    else
        printf("%d\tUser\t", pid[i]);

    printf("%d\t%d\t%d\t%d\t%d\n", at[i], bt[i], ct[i], wt[i], tat[i]);
}
printf("\nAverageWaiting Time: %.2f", total_wt / n);
printf("\nAverageTurnaround Time: %.2f\n", total_tat / n);
printf("\nGanttChart:\n| ");
for(inti=0;i<gindex; i++) {
    if (gantt[i] == -1)
        printf("Idle | ");
```

```
        else
            printf("P%d | ", gantt[i]);
    }
    printf("\n0 ");
    for(inti=1;i<= gindex; i++) {
        printf("%d", i);
    }
    printf("\n");
    return 0;
}
```

OUTPUT:



3.Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

CODE:

```
#include <stdio.h>

#define MAX 10

struct Task {
    int id;
    int execution;
    int period;
    int remaining;
    int deadline;
};

 void RMS(struct Task t[], int n, int timeLimit) {
     printf("\nRate Monotonic Scheduling:\n");

    for (int time = 0; time < timeLimit; time++) {

        for (int i = 0; i < n; i++) {
            if (time % t[i].period == 0)
                t[i].remaining = t[i].execution;
        }

        int selected = -1;

        for (int i = 0; i < n; i++) {
            if (t[i].remaining > 0) {
                if (selected == -1 ||
                    t[i].period < t[selected].period)
                    selected = i;
```

```
            }
        }

        if (selected != -1) {
            printf("Time %d : Task %d\n",
                  time, t[selected].id);
            t[selected].remaining--;
        }
        else {
            printf("Time %d : Idle\n", time);
        }
    }
}

void EDF(struct Task t[], int n, int timeLimit) {
    printf("\nEarliest Deadline First Scheduling:\n");

    for (int time = 0; time < timeLimit; time++) {

        for (int i = 0; i < n; i++) {
            if (time % t[i].period == 0) {
                t[i].remaining = t[i].execution;
                t[i].deadline = time + t[i].period;
            }
        }

        int selected = -1;

        for (int i = 0; i < n; i++) {
             if (t[i].remaining > 0) {
                if (selected == -1 ||
                    t[i].deadline < t[selected].deadline)
                    selected = i;
            }
        }

        if (selected != -1) {
```

```
            printf("Time %d : Task %d\n",
                  time, t[selected].id);
            t[selected].remaining--;
        }
        else {
            printf("Time %d : Idle\n", time);
        }
    }
}

int main() {
    int n, timeLimit;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    struct Task t[MAX], temp[MAX];

    for (int i = 0; i < n; i++) {
        t[i].id = i + 1;

        printf("\nTask %d\n", i + 1);

        printf("Execution Time: ");
        scanf("%d", &t[i].execution);

        printf("Period: ");
        scanf("%d", &t[i].period);

        t[i].remaining = t[i].execution;
        t[i].deadline = t[i].period;

        temp[i] = t[i];
    }

    printf("\nEnter simulation time: ");
    scanf("%d", &timeLimit);
```

```
    RMS(t, n, timeLimit);

    for(int i = 0; i < n; i++)
        t[i] = temp[i];

    EDF(t, n, timeLimit);

    return 0;
}
```

```
Enter number of tasks: 3

Task 1
Execution Time: 1
Period: 4

Task 2
Execution Time: 2
Period: 5

Task 3
Execution Time: 1
Period: 8

Enter simulation time: 10

Rate Monotonic Scheduling:
Time 0 : Task 1
Time 1 : Task 2
Time 2 : Task 2
Time 3 : Task 3
Time 4 : Task 1
Time 5 : Task 2
Time 6 : Task 2
Time 7 : Idle
Time 8 : Task 1
Time 9 : Task 3

Earliest Deadline First Scheduling:
Time 0 : Task 1
Time 1 : Task 2
Time 2 : Task 2
Time 3 : Task 3
Time 4 : Task 1
Time 5 : Task 2
Time 6 : Task 2
Time 7 : Idle
Time 8 : Task 1
Time 9 : Task 3

Process returned 0 (0x0)   execution time : 41.827 s
Press any key to continue.
```

4.Write a C program to simulate:

a) Producer-Consumer problem using semaphores.

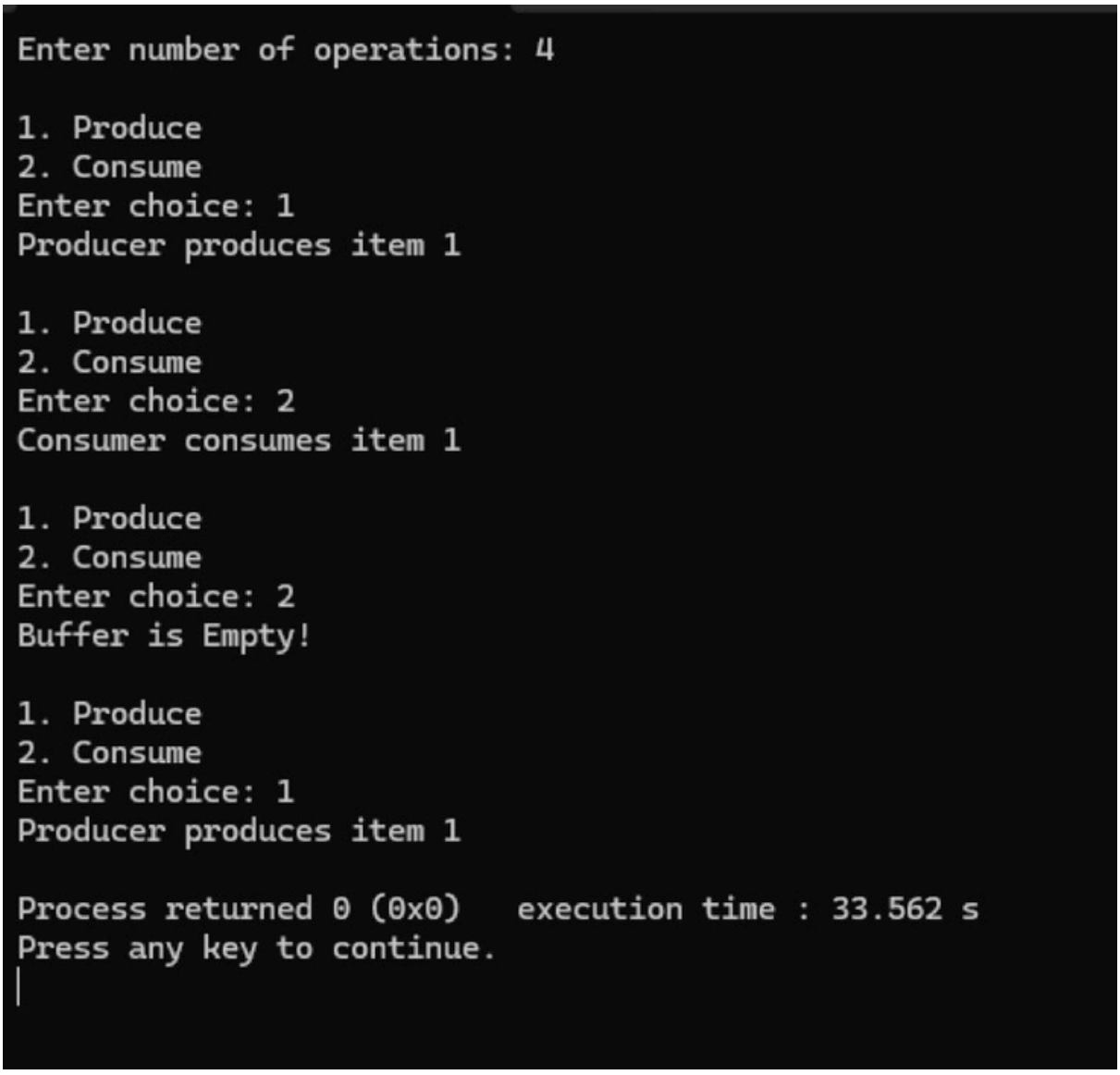
CODE:

```
#include <stdio.h>

#define MAX 10

struct Task {
   int id;
   int execution;
   int period;
   int remaining;
   int deadline;
};

 void RMS(struct Task t[], int n, int timeLimit) {
    printf("\nRate Monotonic Scheduling:\n");

   for (int time = 0; time < timeLimit; time++) {

      for (int i = 0; i < n; i++) {
         if (time % t[i].period == 0)
            t[i].remaining = t[i].execution;
      }

      int selected = -1;

      for (int i = 0; i < n; i++) {
         if (t[i].remaining > 0) {
            if (selected == -1 ||
               t[i].period < t[selected].period)
               selected = i;
         }
      }

      if (selected != -1) {
         printf("Time %d : Task %d\n",
               time, t[selected].id);
         t[selected].remaining--;
      }
      else {
         printf("Time %d : Idle\n", time);
      }
   }
}
```

```
voidEDF(structTaskt[],intn,inttimeLimit) {
   printf("\nEarliestDeadlineFirstScheduling:\n");

   for(inttime=0;time<timeLimit;time++) {

      for (int i = 0; i < n; i++) {
         if (time % t[i].period == 0) {
            t[i].remaining=t[i].execution;
            t[i].deadline=time+t[i].period;
         }
      }

      int selected = -1;

      for (int i = 0; i < n; i++) {
         if (t[i].remaining > 0) {
            if (selected == -1 ||
               t[i].deadline<t[selected].deadline)
               selected = i;
         }
      }

      if (selected != -1) {
         printf("Time %d : Task %d\n",
               time, t[selected].id);
         t[selected].remaining--;
      }
      else {
         printf("Time%d:Idle\n",time);
      }
   }
}

int main() {
   int n, timeLimit;

   printf("Enter number of tasks: ");
   scanf("%d", &n);

   struct Task t[MAX], temp[MAX];

   for (int i = 0; i < n; i++) {
      t[i].id = i + 1;

      printf("\nTask %d\n", i + 1);

      printf("Execution Time: ");
```

```
        scanf("%d", &t[i].execution);

        printf("Period: ");
        scanf("%d", &t[i].period);

        t[i].remaining = t[i].execution;
        t[i].deadline = t[i].period;

        temp[i] = t[i];
    }

    printf("\nEnter simulation time: ");
    scanf("%d", &timeLimit);

    RMS(t, n, timeLimit);

    for(int i = 0; i < n; i++)
        t[i] = temp[i];

    EDF(t, n, timeLimit);

    return 0;
}
```

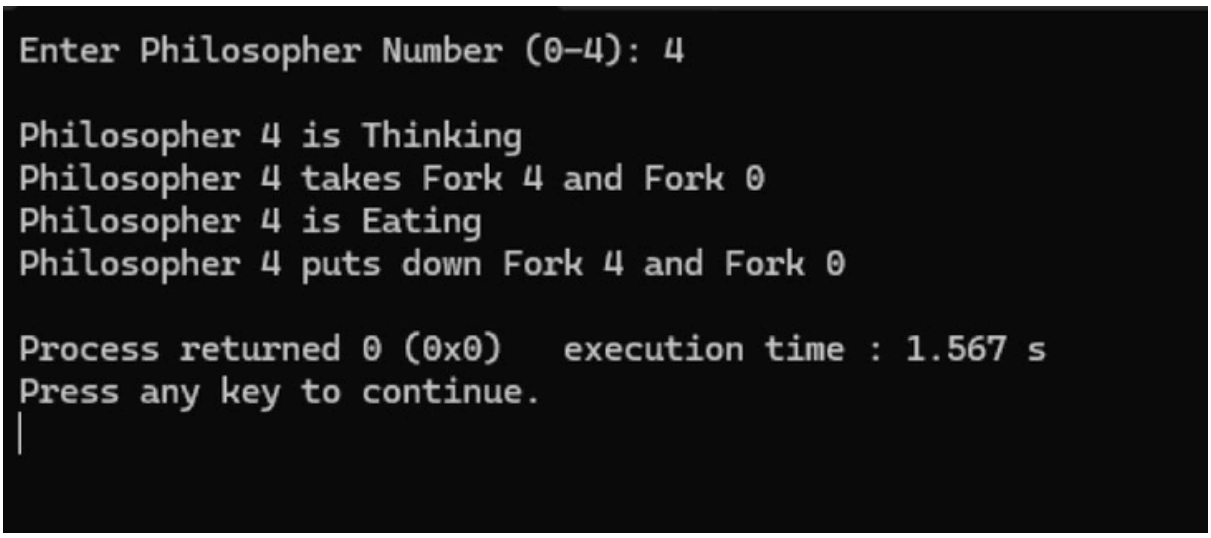
OUTPUT:


Enter number of operations: 4
1. Produce
2. Consume
Enter choice: 1
Producer produces item 1
1. Produce
2. Consume
Enter choice: 2
Consumer consumes item 1
1. Produce
2. Consume
Enter choice: 2
Buffer is Empty!
1. Produce
2. Consume
Enter choice: 1
Producer produces item 1
Process returned 0 (0x0) execution time : 33.562 s
Press any key to continue.

b) Dining-Philosopher's problem

```
#include <stdio.h>

#define N 5

void dining(int ph)
{
   printf("\nPhilosopher%d is Thinking", ph);

   printf("\nPhilosopher%d takes Fork %d and Fork %d",
         ph,ph,(ph+1)%N);

   printf("\nPhilosopher%d is Eating", ph);

   printf("\nPhilosopher%d puts down Fork %d and Fork %d\n",
         ph,ph,(ph+1)%N);
}

int main()
{
   int ph;

   printf("EnterPhilosopher Number (0-4): ");
   scanf("%d", &ph);

   if(ph >= 0 && ph < N)
      dining(ph);
   else
      printf("InvalidPhilosopher Number");

   return 0;
}
```

OUTPUT:


Enter Philosopher Number (0-4): 4
Philosopher 4 is Thinking
Philosopher 4 takes Fork 4 and Fork 0
Philosopher 4 is Eating
Philosopher 4 puts down Fork 4 and Fork 0
Process returned 0 (0x0) execution time : 1.567 s
Press any key to continue.

5.Write a C program to simulate:

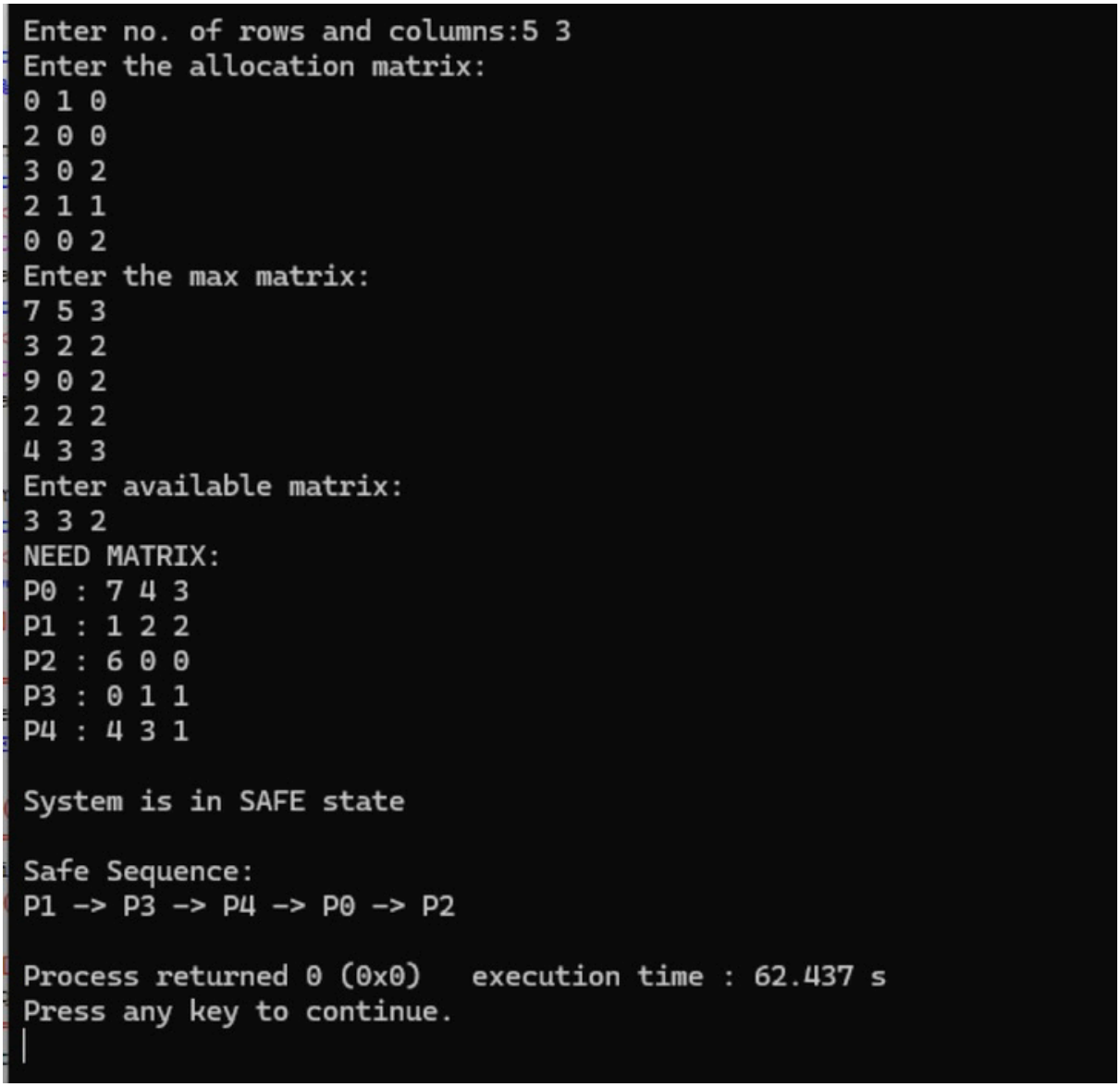
a)Bankers' algorithm for the purpose of deadlock avoidance.

CODE:

```
#include <stdio.h>

int main() {
   int n,m;
   printf("Enter no. of rows and columns:");
   scanf("%d %d",&n,&m);
   int i,j,k;
   int alloc[n][m], max[n][m];
   printf("Enter the allocation matrix:\n");
   for (i=0;i<n;i++)
      for(j=0;j<m;j++)
         scanf("%d",&alloc[i][j]);
   printf("Enter the max matrix:\n");
   for (i=0;i<n;i++){
      for(j=0;j<m;j++){
         scanf("%d",&max[i][j]);
      }
   }
   int avail[m];
   printf("Enter available matrix:\n");
   for (i=0;i<m;i++)
      scanf("%d", &avail[i]);
   int need[n][m];
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         need[i][j] = max[i][j] - alloc[i][j];
   printf("NEED MATRIX:\n");
   for(i = 0; i < n; i++) {
      printf("P%d : ", i);
      for(j = 0; j < m; j++)
         printf("%d ", need[i][j]);
      printf("\n");
   }
   int finish[5] = {0};
   int safeSeq[5];
   int count = 0;
   while(count < n) {
   int found = 0;
   for(i = 0; i < n; i++) {
         if(finish[i] == 0) {
            for(j = 0; j < m; j++) {
```

```
            if(need[i][j]>avail[j]) break;
        }
        if(j == m) {
            for(k=0;k<m; k++)
                avail[k]+=alloc[i][k];
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
      }
    }
    if(found == 0) {
      printf("\nSystemisNOT in safe state\n");
      return 0;
    }
  }
  printf("\nSystemisinSAFE state\n");
  printf("\nSafeSequence:\n");
  for(i = 0; i < n; i++) {
    printf("P%d",safeSeq[i]);
    if(i != n - 1)
      printf(" -> ");
  }
  printf("\n");
  return 0;
}
```

OUTPUT:

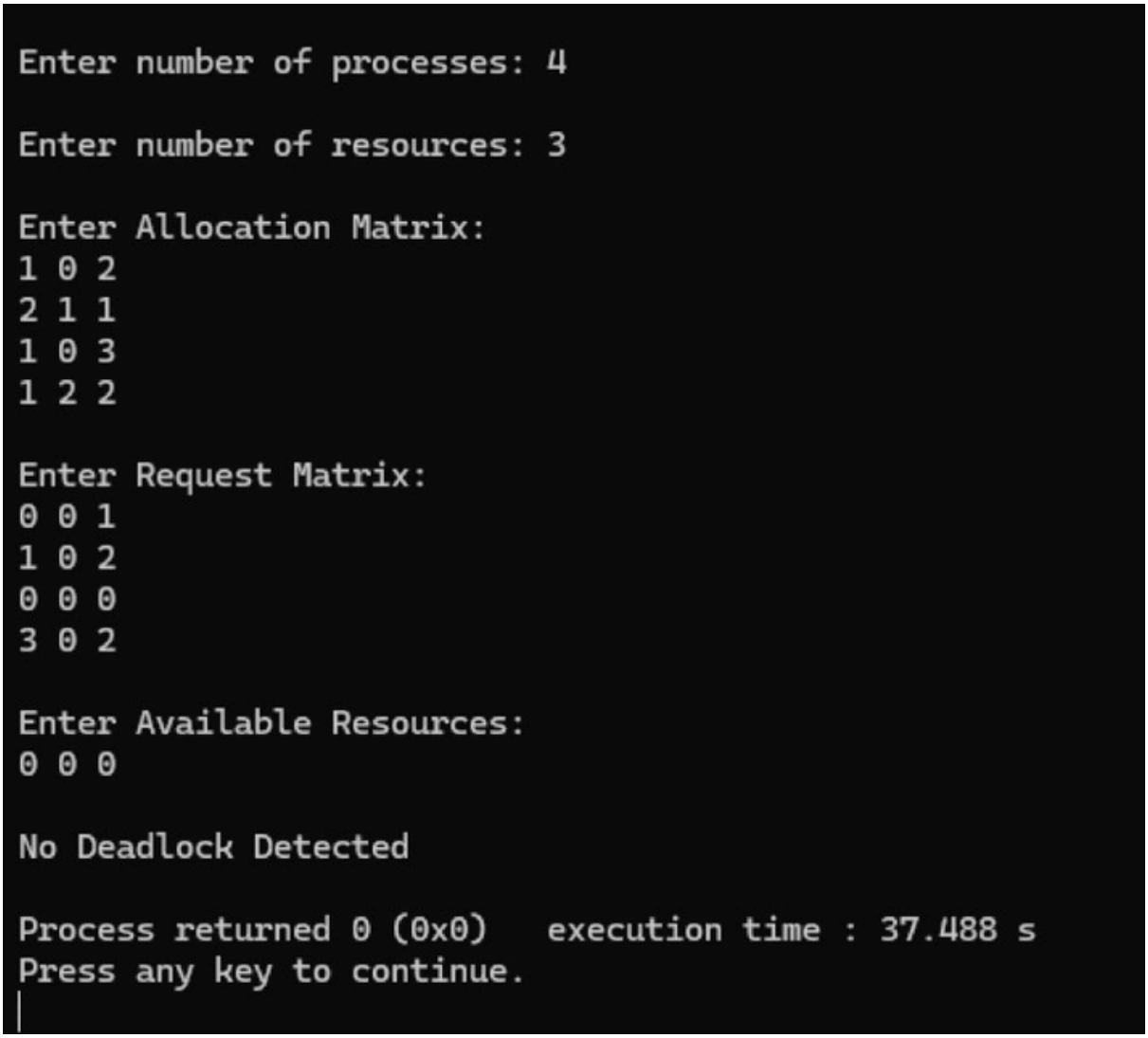

Enter no. of rows and columns:5 3
Enter the allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter the max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter available matrix:
3 3 2
NEED MATRIX:
P0 : 7 4 3
P1 : 1 2 2
P2 : 6 0 0
P3 : 0 1 1
P4 : 4 3 1
System is in SAFE state
Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0) execution time : 62.437 s
Press any key to continue.

b) Deadlock Detection

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("\nEnter number of processes: ");
    scanf("%d", &n);
    printf("\nEnter number of resources: ");
    scanf("%d", &m);
    int alloc[n][m], request[n][m];
    int avail[m];
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
    scanf("%d", &alloc[i][j]);
    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &request[i][j]);
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);
    int finish[n];
    for(i = 0; i < n; i++) {
        int empty = 1;
        for(j = 0; j < m; j++) {
            if(alloc[i][j] != 0) {
                empty = 0;
                break;
            }
        }
        finish[i] = empty;
    }
    int found;
    do {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                for(j = 0; j < m; j++) {
                    if(request[i][j] > avail[j])
                        break;
                }
                if(j == m) {
                    for(k = 0; k < m; k++)
                        avail[k] += alloc[i][k];
```

```
                finish[i] = 1;
                found = 1;
            }
        }
    }

}while(found);
intdeadlock = 0;
for(i=0;i< n; i++) {
    if(!finish[i]) {
        deadlock = 1;
        printf("\nProcess P%d is in deadlock\n", i);
    }
}
if(!deadlock)
    printf("\nNo Deadlock Detected\n");
return 0;
}
```

OUTPUT:


Enter number of processes: 4
Enter number of resources: 3
Enter Allocation Matrix:
1 0 2
2 1 1
1 0 3
1 2 2
Enter Request Matrix:
0 0 1
1 0 2
0 0 0
3 0 2
Enter Available Resources:
0 0 0
No Deadlock Detected
Process returned 0 (0x0) execution time : 37.488 s
Press any key to continue.

6. WriteaCprogramtosimulate the following contiguous memory allocation techniques.

a) Worst-fit
b) Best-fit
c) First-fit

CODE:

```
#include <stdio.h>

#define MAX 25

voidfirstFit(intblockSize[], int m, int processSize[], int n) {
   int allocation[MAX];
   int temp[MAX];

   for(inti=0;i<m;i++)
      temp[i]=blockSize[i];

   for(inti=0;i<n;i++)
      allocation[i]= -1;

   for(inti=0;i<n;i++) {
      for(intj=0;j<m;j++) {
         if(temp[j]>=processSize[i]) {
            allocation[i]=j;
            temp[j]-=processSize[i];
            break;
         }
      }
   }

   printf("\n---FirstFit---\n");
   printf("ProcessNo.\tProcess Size\tBlock No.\n");

   for(inti=0;i<n;i++) {
      printf("%d\t\t%d\t\t", i + 1, processSize[i]);
```

```
        if(allocation[i]!=-1)
            printf("%d\n",allocation[i] + 1);
        else
            printf("NotAllocated\n");
    }
}

voidbestFit(intblockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int temp[MAX];

    for(inti=0;i<m;i++)
        temp[i]=blockSize[i];

    for(inti=0;i<n;i++)
        allocation[i]= -1;

    for(inti=0;i<n;i++) {
        int bestIdx= -1;

        for(intj=0;j<m;j++) {
            if(temp[j]>=processSize[i]) {
                if(bestIdx==-1 || temp[j] < temp[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i]=bestIdx;
            temp[bestIdx]-=processSize[i];
        }
    }

    printf("\n---BestFit---\n");
    printf("ProcessNo.\tProcess Size\tBlock No.\n");
```

```
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t",i+1, processSize[i]);

        if(allocation[i]!= -1)
            printf("%d\n",allocation[i] + 1);
        else
            printf("NotAllocated\n");
    }
}

voidworstFit(intblockSize[],int m, int processSize[], int n) {
    int allocation[MAX];
    int temp[MAX];

    for (int i = 0; i < m; i++)
        temp[i] = blockSize[i];

    for (int i = 0; i < n; i++)
        allocation[i]= -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;

        for(intj=0;j<m;j++){
            if(temp[j]>=processSize[i]) {
                if(worstIdx==-1||temp[j] > temp[worstIdx]) {
                    worstIdx = j;
                }
            }
        }

        if(worstIdx!= -1){
            allocation[i]=worstIdx;
            temp[worstIdx]-=processSize[i];
        }
    }
```

```
    printf("\n--- Worst Fit ---\n");
    printf("ProcessNo.\tProcessSize\tBlock No.\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t",i+1,processSize[i]);

        if(allocation[i]!= -1)
            printf("%d\n",allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    intblockSize[MAX],processSize[MAX];
    int m, n;

    printf("Enternumberofmemoryblocks: ");
    scanf("%d", &m);

    printf("Entersizesof%dmemoryblocks:\n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
    }

    printf("Enternumberofprocesses:");
    scanf("%d", &n);

    printf("Entersizesof%dprocesses:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    firstFit(blockSize,m,processSize,n);
    bestFit(blockSize,m,processSize,n);
    worstFit(blockSize,m,processSize,n);
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
200 300 400 500 600
Enter number of processes: 4
Enter sizes of 4 processes:
110 120 130 140

--- First Fit ---
Process No.     Process Size    Block No.
1               110             1
2               120             2
3               130             2
4               140             3

--- Best Fit ---
Process No.     Process Size    Block No.
1               110             1
2               120             2
3               130             2
4               140             3

--- Worst Fit ---
Process No.     Process Size    Block No.
1               110             5
2               120             4
3               130             5
4               140             3

Process returned 0 (0x0)   execution time : 28.100 s
Press any key to continue.
```

7. Write a C program to simulate page replacement algorithms.
a) FIFO
b) LRU
c) Optimal

CODE:

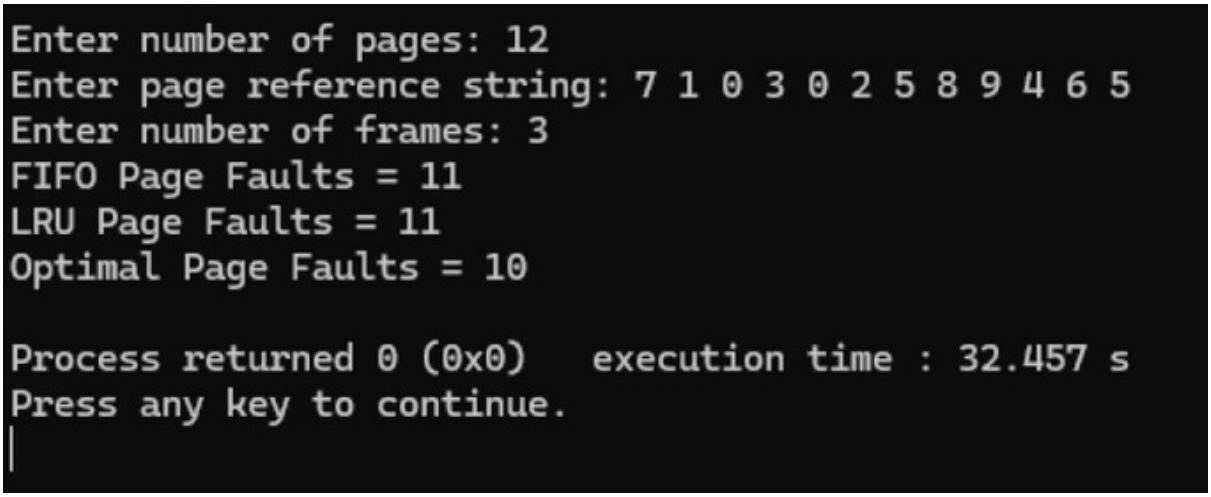
```
#include <stdio.h>
int frames[10];
void fifo(int pages[], int n, int f) {
    int i, j, k = 0, fault = 0, found;
    for(i = 0; i < f; i++)
    frames[i] = -1;
    for(i = 0; i < n; i++) {
        found = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if(!found) {
            frames[k] = pages[i];
            k = (k + 1) % f;
            fault++;
        }
    }
    printf("FIFO Page Faults = %d\n", fault);
}
void lru(int pages[], int n, int f) {
    int i, j, least, pos, fault = 0;
    int time[10], counter = 0;
    for(i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;
    }
    for(i = 0; i < n; i++) {
        int found = 0;
```

```
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
        if(!found) {
            least = time[0];
            pos = 0;
            for(j = 0; j < f; j++) {
                if(frames[j] == -1) {
                    pos = j;
                    break;
                }
                if(time[j] < least) {
                    least = time[j];
                    pos = j;
                }
            }
            frames[pos] = pages[i];
            counter++;
            time[pos] = counter;
            fault++;
        }
    }
    printf("LRU Page Faults = %d\n", fault);
}
void optimal(int pages[], int n, int f) {
int i, j, k, pos, farthest, fault = 0;
for(i = 0; i < f; i++)
frames[i] = -1;
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
```

```
            found = 1;
            break;
        }
    }
    if(!found){
        pos = -1;
        for(j = 0; j < f; j++) {
            if(frames[j] == -1) {
                pos = j;
                break;
            }
        }
        if(pos == -1) {
            farthest =i;
            pos = 0;
            for(j = 0; j < f; j++) {
                int next = -1;
                for(k=i+1;k<n;k++) {
                    if(frames[j]==pages[k]) {
                        next = k;
                        break;
                    }
                }
                if(next == -1) {
                    pos = j;
                    break;
                }
                if(next > farthest) {
                    farthest = next;
                    pos = j;
                }
            }
        }
        frames[pos] = pages[i];
        fault++;
    }
}
```

```
    printf("Optimal Page Faults = %d\n", fault);
}
intmain() {
    intpages[20], n, f, i;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    fifo(pages, n, f);
    lru(pages, n, f);
    optimal(pages, n, f);
    return 0;
}
```

OUTPUT:


Enter number of pages: 12
Enter page reference string: 7 1 0 3 0 2 5 8 9 4 6 5
Enter number of frames: 3
FIFO Page Faults = 11
LRU Page Faults = 11
Optimal Page Faults = 10
Process returned 0 (0x0) execution time : 32.457 s
Press any key to continue.